Username-less Passkey Authentication

This document describes a web app with a username-less passkey authentication.

For context, the web app is named `rustctl`, and it's a *Rust* (the survival video game distributed on *Steam*) server management and observability tool.

1 Authentication And Authorization

To authenticate, a web browser client first creates a passkey and associates it with e.g. domain `rustctl.internal` in its platform store (e.g. wherever *Windows 11* stores passkeys for *Brave* browser to use). Then, the client sends the passkey's public part to the server, and the server stores the passkey. At this point the server knows the passkey as just some passkey that is not yet associated with any user that would be specifically authorized to do anything. The server sets a signed cookie for the client asserting that the session is associated with the specific passkey's public key. This process is depicted in Figure 1.

To be authorized to do something (like set some in-game parameters of the manageable Rust game server), a client session needs to associate itself with a Steam identity. That is, a client who already has a passkey associated with its session must link the session with a Steam account. This is implemented the usual OAuth-like way (call it delegated authentication or whatever) that Steam provides. Once this is done, the client session is associated with both a passkey and a Steam ID. The server then checks the Steam ID to determine what the client is allowed to do when the client requests to do something.

To further emphasize, this design implies anonymous users in some sense. That is, the passkey itself serves as an identity, and multiple different passkeys (e.g. across multiple devices with non-synchronized passkey stores) may be later associated with a specific Steam ID. A non-anonymous, i.e. a well defined user (in the sense that one may be authorized to do stuff), only emerges once a Steam ID has been associated with one or more passkeys.

The passkey specification forces to define a `name`, a `displayName` and an `id` for a `user`, but what those really mean in this system are names and an ID for the passkey, and not for a user (because again, the concept of a user in this system emerges from an association with a Steam ID). Passkeys are also associated with a credential ID, and therefore in this system each passkey is associated with two IDs: the thing that is specified as the aforementioned `user.id` and the thing that is referred to as credential ID. In this system, there is not semantic difference between the two IDs despite them having different values and being stored in different places.

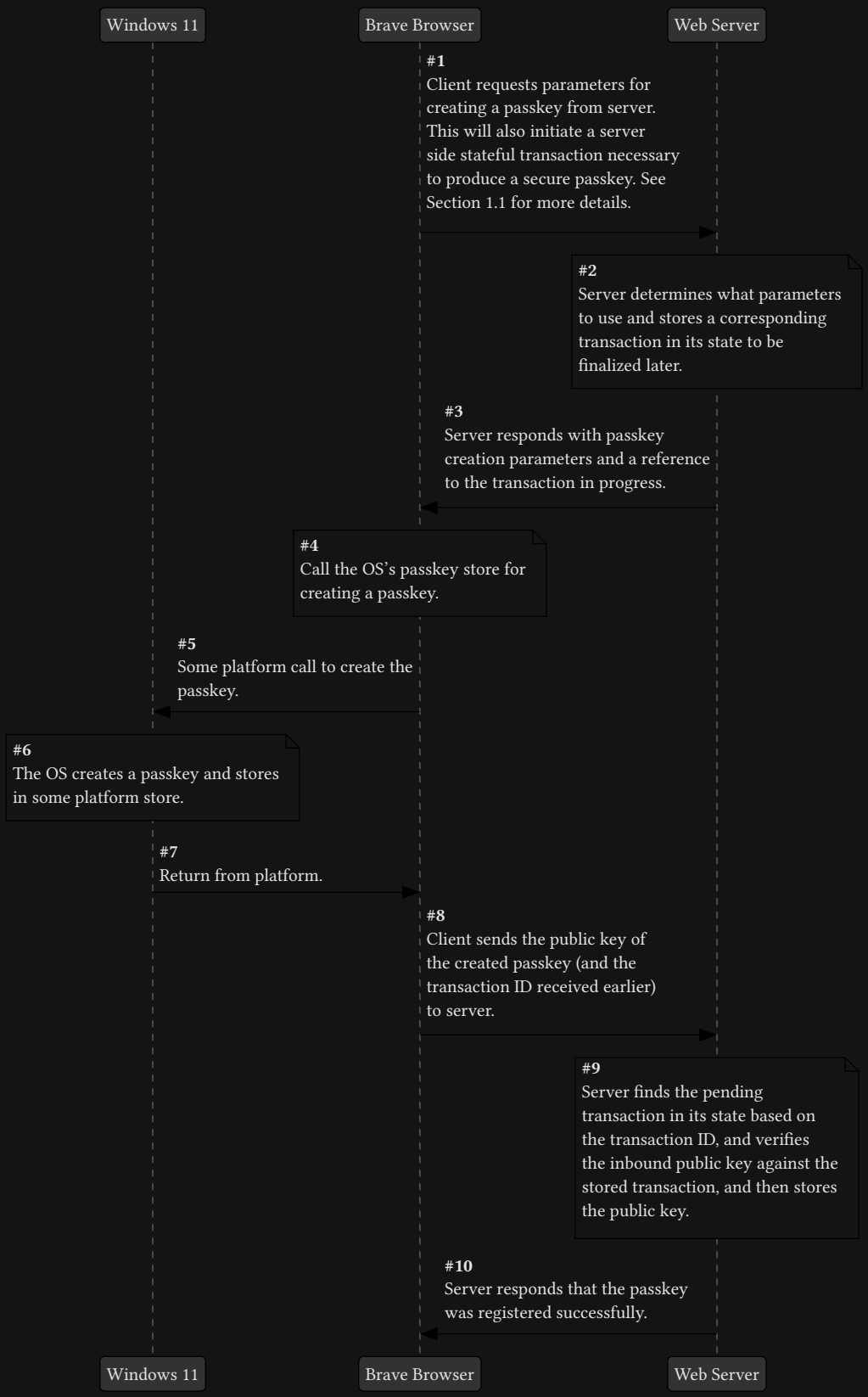

Figure 1: Overview of creating a passkey.

1.1 Passkey Registration On Server

The server-side transaction exists to make passkey registration a well-defined challenge-response protocol rather than a mere public-key upload. When the server initiates passkey creation, it commits to a specific set of parameters (in particular a fresh, unpredictable challenge and the RP (Relying Party) ID under which the credential is to be created) and records them temporarily as a pending transaction. When the client later returns with the newly created credential, the server uses this stored transaction to cryptographically verify that the credential was created in direct response to its challenge, under the expected origin and policy, and within an acceptable time window. This prevents replay, substitution, and cross-origin attacks, and ensures that only credentials explicitly authorized by the server become registered.